

CryoDRGN Method Research

Erica Wang, Sreeyutha Ratala, Anna Szczuka

December 2024

1 Introduction

Cryo-electron microscopy (cryo-EM) [5] is a cutting-edge technique to study minute details on protein structures at a near-atomic resolution (surpassing previous state-of-the-art methods such as X-ray crystallography and nuclear magnetic resonance). The target protein is frozen in a layer of vitreous ice to preserve the structure of the protein from damage as its molecules are bombarded with electrons to create a projection image. The inverse reconstruction of the 3D volume of the protein complex from noisy 2D cryo-em images is a challenging but rewarding task. Target proteins exhibit unknown heterogeneities in their orientation (random 2D translation and rotation), conformation, and composition. Conformational heterogeneities stem from the inherent flexibility and rotational movement of internal subunits, whereas compositional heterogeneities arise from differences in the molecular composition or the presence of varying structural components. Combined with a low signal-to-noise-ratio (SNR) caused by radiation damage on the sample, the task is further complicated.

In order to recover a 3D protein reconstruction from 2D cryo-EM images, reconstruction must solve for the protein volume structure and the pose. Traditional cryo-EM reconstruction methods, such as refinement methods like expectation maximization and variants of coordinate ascent, are very sensitive to initial estimates of protein volume. Additionally, current approaches to resolving structural heterogeneity in cryo-EM work well with a limited number of discrete conformations and require manual selection of images in a dataset to match protein structural or conformational states. To address the error proneness of traditional cryo-EM reconstruction methods, we can employ machine learning to learn a function that maps input cryo-EM projections to protein volumes. Machine learning can disentangle image content from spatial transformations (such as rotations and translations) by enabling independent representations of image features and spatial positioning through continuous latent variable models like deep neural networks. Spatial-VAE [1], a novel variational autoencoder framework, includes spatial coordinates (x, y) as input to a function that outputs a distribution of pixel intensities, enabling rotational and translational variations to be learned independently from the intrinsic image content. Rotational and translational latent representations are backpropagated as variables passed into the decoder in the training loop.

A state-of-the-art extension of the Spatial-VAE, CryoDRGN [9], integrates exact inference over orientation heterogeneity with an extended Spatial-VAE encoder-decoder network to achieve unsupervised reconstruction of a continuous low-dimensional representation of the protein’s conformational state. A Fourier transform [6] is applied to the 2D cryo-EM images to simplify the reconstruction problem, noting the Fourier slice Theorem. This project aimed to investigate how cryo-EM images can be reconstructed into continuous ensembles of protein structures.

2 Background

2.1 Dataset

Our dataset contains a hundred protein volumes with corresponding synthetically generated cryo-EM images. Volumes were obtained from the CryoBench dataset [3], which provides a high-quality labeled reserve of 2D cryo-EM images covering a range of resolution, noise levels, and types of conformational and compositional complexities. For our experiments, we chose to generate our own Cryo-EM images from these volumes. Most importantly, our images are noiseless.

Our dataset for each volume consists of n images, $I_1, \dots, I_n$, where $I_i \in \mathbb{R}^{D \times D}$. Each image is generated with the following image formation model:

$$I_i(x, y) = \int_{-\infty}^{+\infty} \rho(R_i r) dz, \quad \mathbf{r} = (x, y, z)^T$$

Let $\rho : \mathbb{R}^3 \rightarrow \mathbb{R}$ describe the 3D density of a protein. Each point (x, y, z) in the volume is rotated by a 3 x 3 rotation matrix R_i , and $\rho(R_i(x, y, z))$ represents the density at that rotated point. R_i is a rotation matrix corresponding to some $\theta \in (0, 2\pi)$ applied to a random rotation axis. After the protein is rotated, we project (taking a line integral) along a set axis.

The fundamental cryo-EM problem, restated, is to predict ρ given $I_1, \dots, I_n$, when R_i are unknown. One method to reconstruct the full 3D volume from such 2D projections requires reversing the line integration in the image construction model to recover the density values across the entire volume. This inverse process is computationally intensive, as it involves high-dimensional integration to combine cumulative contributions from projections across multiple viewing angles, ultimately solving for each voxel in the 3D structure. An improved method utilizes the Fourier Slice Theorem.

2.2 Fourier Slice Theorem

The Fourier slice theorem [6] provides a nifty trick to this problem. The Fourier slice theorem asserts that the Fourier transform of a 2D projection is equivalent to a slice taken through a Fourier transform of the corresponding 3D volume. Mathematically, the previous equation can be expressed in the Fourier domain.

$$\hat{I}(k_x, k_y) = \hat{\rho}(R_i(\cdot, \cdot, 0))$$

$\hat{\cdot}$ denotes the Fourier space. k_x, k_y are coordinates in the Fourier domain. $(\cdot, \cdot, 0)$ indicates taking a slice along the $z = 0$ plane. Applying a rotation matrix R_i to it denotes taking a slice along the plane rotated by this rotation matrix. Now that we are working in the Fourier space when considering the 3D reconstruction problem, we can gather projections from various rotations and obtain slices of the 3D Fourier transform that can be aligned and arranged into a 3D Fourier grid, effectively bypassing the need for complex integrals in real space.

2.3 Fourier Slice Theorem Proof

To prove the Fourier Slice Theorem, we compare the Fourier transform of a projection of a rotated volume to the rotated slice of the Fourier transform of said volume.

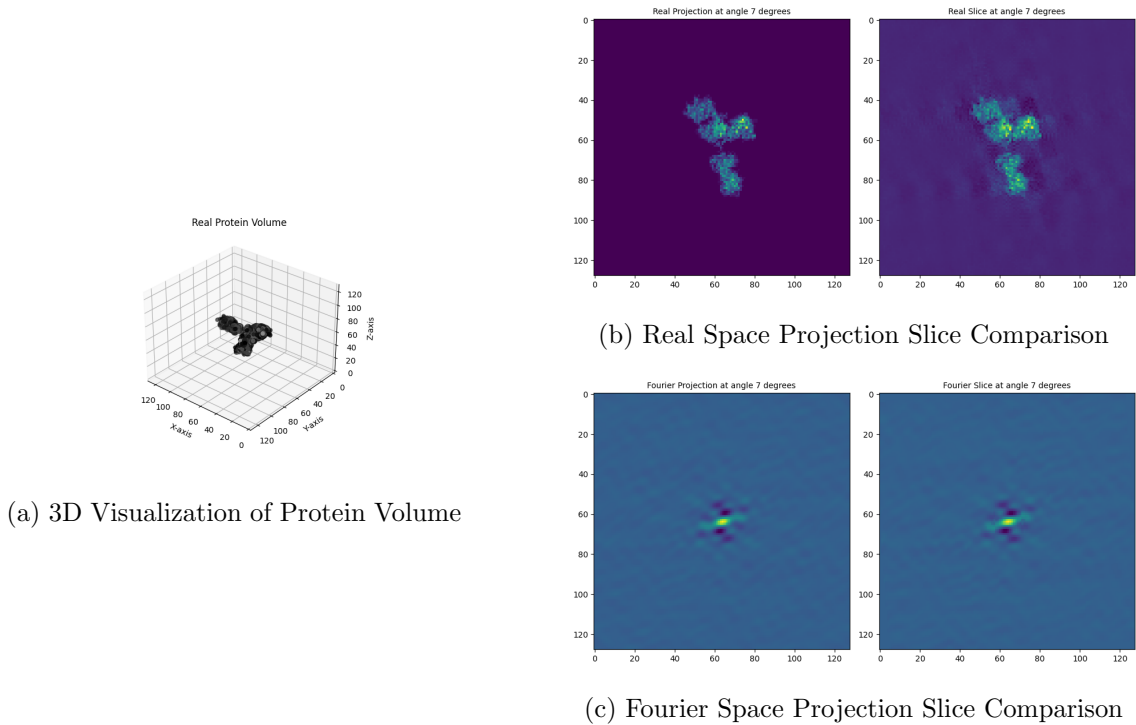


Figure 1: Data Visualizations for a protein in our dataset. a) We begin by visualizing the 3D protein in real space b) Given a 3D protein volume, we take a projection of the rotated ($\theta = 7$ degrees) volume and a slice of the Fourier transformed volume at the same rotation. Inverse Fourier transforming this slice gives a real slice that is compared to the original projection. c) Given the projection, we Fourier transform it and compare it to the Fourier space slice.

Figures 1b) - 1d) prove the Fourier Slice Theorem. Given a 3D volume of shape 128 x 128 x 128, we take a 2D projection along a rotation axis with a line integral from the Radon transformation. This is plotted in Figure 1b) left. If we take its Fourier transform, we get plot Figure 1c) left. 1c) right plots a 2D slice of the Fourier transform directly applied to the 3D volume. Images c) left and c) right are very similar, confirming the Fourier Slice Theorem, which states that a Fourier transform of a 2D projection using the Radon transform of a 3D volume is equivalent to the slice of a 3D Fourier transform on the 3D volume along the projection angle.

*Notes on Radon transform: A Radon transform applied on a 3D space defines a projection plane according to the xy-axis (after a rotation is applied). For each point on this xy-plane, the transform sums up or “integrates” the values of $f(x,y,z)$ along the entire line perpendicular to the point. This sum is called the “intensity” along the line. The final product is a 2D projection along an orientation axis.

3 Methods

Our overall goal was to implement a model that follows the novel architecture of the CryoDRGN model and reconstructs continuous distributions of 3D protein structure from 2D projections of 3D proteins. The pipeline can be separated into two major components: the machine learning side (the MLP-based model architecture that outputs 2D slices) and the reconstruction task (the reconstruction of the outputted 2D slice into a 3D protein structure).

3.1 CryoDRGN

3.1.1 Objective & Pipeline

We utilize cryoDRGN (Deep Reconstructing Generative Networks) [9] for *ab initio* reconstruction of 3D protein structures from multiple 2D views captured at unknown orientations. CryoDRGN builds on the Spatial Variational Autoencoder (VAE) framework [1], where the encoder learns a latent representation, z , that captures the protein’s structural variability (heterogeneity), while the decoder reconstructs the 3D volume at any specified spatial coordinate. Unlike conventional VAEs, which encode all sources of heterogeneity into a single latent variable, cryoDRGN separates the extrinsic orientation of the volume (an imaging artifact) from its intrinsic heterogeneity (a biological feature), providing a more accurate representation of the underlying structural dynamics.

Our end-to-end training framework directly relates 2D views to 3D volumes by modeling the process through which 2D projections are generated from 3D volumes. For inference, we use a variational approach to represent unconstrained latent variables and estimate the unknown pose of each 2D projection by separating rotational orientations from other latent variables learned. Alternatively, we can perform a global search over the space of all possible poses, $SO(3) \times \mathbb{R}^2$, to determine the orientation of each 2D image in $\mathbb{R}^3$. The details of these components are discussed in the following sections.

3.1.2 Generative Reconstruction Model

CryoDRGN is a deep generative model that approximates a *single* function, $\hat{V}$, mapping 3D coordinates in $\mathbb{R}^3$ to electron densities in $\mathbb{R}$. This function operates in Fourier space¹, where the Fourier Slice theorem applies. Specifically, the decoder $p_\theta(\hat{V} | k, z)$ models the conditional distribution of electron density values given a point $k \in \mathbb{R}^3$ in Fourier space and a latent variable z that captures the pose-invariant representation of the protein’s intrinsic structural variations. Unlike standard VAE decoders, which learn a *separate* function for each voxel in a D^3 lattice, CryoDRGN models a single continuous function over Cartesian coordinates and the latent variable, similar to the Spatial VAE. Since the decoder’s output represents electron densities at specific pixel locations in 3D Fourier space, its output for 2D lattices can directly relate to the 3D protein volume through the Fourier Slice theorem.

Recall that our 2D views are generated by projecting the 3D volume of the protein along a randomly chosen axis. In the Fourier domain, the Fourier transform of each 2D view corresponds to an oriented central slice of the 3D volume (see the Fourier Slice Theorem section). During training, we determine the 3D coordinates corresponding to the pixels of the 2D image by applying a rotation to a $D \times D$ lattice initially defined on the $x-y$ plane.

In some experiments, instead of directly providing the decoder with the pixel location k in the Fourier domain, we use a positional encoding of k , composed of sine and cosine waves at varying frequencies. These encodings share the same dimensionality, d_z , as the latent variable z , allowing them to be either summed or concatenated

¹Since our model Hartley space is an alternative domain where the Fourier Slice theorem holds, allowing computations without involving complex numbers.

with z along the last dimension. Specifically, we employ the fixed positional encoding from [7], applied to the spatial coordinates $\mathbf{k}_j \in \mathbf{k}$, which is defined as:

$$\begin{aligned} \text{PE}_{2i}(\mathbf{k}_j) &= \sin\left(\frac{\mathbf{k}_j}{10000^{\frac{2i}{d_z}}}\right), \quad i = 1, \dots, \frac{d_z}{2}; \quad \mathbf{k}_j \in \mathbf{k} \\ \text{PE}_{2i+1}(\mathbf{k}_j) &= \cos\left(\frac{\mathbf{k}_j}{10000^{\frac{2i}{d_z}}}\right), \quad i = 1, \dots, \frac{d_z}{2}; \quad \mathbf{k}_j \in \mathbf{k} \end{aligned}$$

3.1.3 Model Inference

We utilize a conventional VAE to approximate the latent variable z

Variational Encoder: The image encoder is designed to capture a pose-invariant representation of a protein’s intrinsic structural heterogeneity, encompassing variations in shape, folding, and other conformations that arise while the protein performs its biological function. This pose-invariance is achieved because the decoder incorporates spatial coordinates that encode the rotation of the volume corresponding to the input image, eliminating the need for z to capture this information. Following the standard VAE framework, a cryo-EM image $\hat{X}$, represented as a $D \times D$ vector, is passed into the probabilistic encoder $q_\xi(z | \hat{X})$, implemented as an MLP parameterized by ξ . The encoder outputs the mean $\mu_{z|\hat{X}}$ and covariance $\Sigma_{z|\hat{X}}$, which define an approximate posterior distribution for sampling z (whose prior is a standard normal distribution, $\mathcal{N}(0, I)$).

From Spatial VAE, this decoder models the distribution of density based on the Cartesian coordinates (in contrast to the standard VAE). The decoder outputs $p_\theta(\hat{V} | k, z)$ such that when given the positional encoding $k \in \mathbb{R}^3$ and latent variable z , we find the Gaussian distribution at $\hat{V}(k, z)$ - this is the distribution of density of $\hat{V}_z$ at frequency k in Fourier space.

Pose Inference: CryoDRGN does not attempt to learn the rotations, so to address the challenge of producing unsupervised reconstructions of 3D protein structures when the poses of protein projections are unknown, we extend the CryoDRGN model to include pose learning. Specifically, we modify the architecture to extract the last four components of a sampled latent representation of the input image, encoding the rotation using quaternions. This addition enables the model to infer the rotation of the input image while maintaining the original decoder architecture. Theoretically, this approach aligns with the spatial-VAE framework, offering the capability to learn true rotations for each input image and achieve multi-view consistency. In principle, any object that remains internally unchanged will produce the same projected view under the same rotation along a consistent axis. If the model predicts an incorrect rotation during training, the decoder’s reconstructed slice (derived from the learned 3D volume) will differ from the input slice. This discrepancy introduces a penalty via the reconstruction loss, driving the model toward learning the true rotation.

Over many training iterations, this iterative correction ensures that the model stochastically converges on the correct rotations. Any incorrect rotation leads to inconsistencies with the other dataset elements, making it less likely to persist as training progresses. Consequently, the model gradually aligns its learned rotations to ensure consistency across all views, enabling accurate 3D reconstructions from unsupervised 2D projections.

Specifically, when training, we aim to find an optimal θ such that the distributions for the density outputted given the positional information and latent variable help reconstruct the image. This is an extension of the spatial VAE model. Unlike a regular VAE, spatial VAEs associate a positional encoding with the image such that it learns to reconstruct an image with explicit consideration of spatial positions, allowing us to separate the structural heterogeneities from rotational and translational factors and better understand the structural information at each pixel. Under the spatial VAE approach, the pose is encoded in a component of the latent variable z . Then, the designated components of the latent variable z corresponding to the learned pose are separated from z and used to rotate the input slice grid points into the decoder. The mapping allows the model to encode both the visual features and the spatial configuration of the protein. One way to encode pose in the latent variable z is through a quaternion. The first four latent components of z correspond to the components of the quaternion (representing the learned rotation of the volume corresponding to the 2D input image in the case). We convert the quaternion into a rotation matrix that is applied to the input grid to the decoder. Since we query the decoder at the slice points with the relevant rotations, the decoder must stick the correct slice at every point we rotate into. When all the points are queried eventually, multi-view consistency enables the model to try to stitch the appropriate slice at every rotated point. Another method we implemented to learn the rotation was a MLP that takes the latent z as input and outputs values corresponding to the quaternion that is converted to a rotation matrix. The prior is Gaussian. We note that CryoDRGN does not learn the rotations through this framework and instead

uses a Branch & Bound search algorithm. Multi-view consistency is established in 3D deep learning.

To summarize, the training is as follows.

1. Given a 2D image $\hat{X}_i$, the encoder outputs the learned posterior: $\mu_{z|X_i}$ and $\Sigma_{z|X_i}$.
2. A value of the latent is sampled from the posterior; i.e., $z_i \sim \mathcal{N}(\mu_{z|X_i}, \Sigma_{z|X_i})$.
3. In the case of unsupervised rotation training, the first four components of z_i correspond to the quaternion and are converted to a 3×3 rotation matrix R .
4. The coordinates of the true locations of each pixel of the input image on the 3D volume in Fourier space are computed by multiplying R^T with each coordinate in a $D \times D$ lattice (i.e., a grid of equally spaced coordinates ranging from -0.5 to 0.5 on the xy plane at $z = 0$).
5. z_i is broadcast to the positional encoding of every coordinate (specified above) and passed as input to the decoder.
6. The decoder outputs the electron density for each coordinate passed in, thereby reconstructing the original image pixel by pixel using the spatial information.
7. Since z_i is pose-invariant as we extract the rotation out of z_i , the reconstruction loss of the decoder's output image will be very high unless the correct rotation is learned to achieve multi-view consistency.

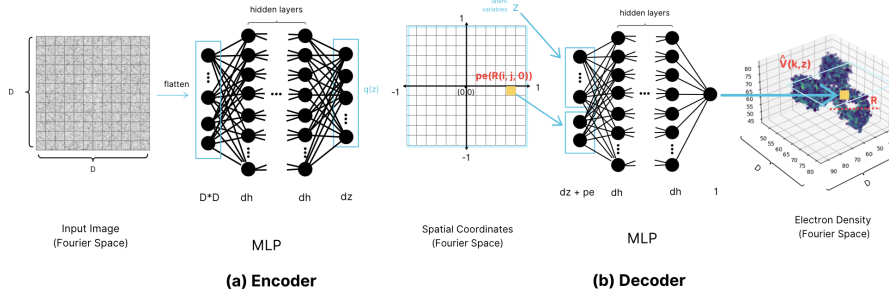


Figure 2: Pipeline of the CryoDRGN [9] method. Input images are passed to the encoder, which outputs a distribution of the latent variable z . Using the Branch and Bound Algorithm, the pose of the image is found. A positional encoding of each pixel is created. This positional encoding, along with sampled values of z , is passed to the decoder, which gives us a slice of the 3D electron density in Fourier space.

Given some input image, we translate it into Fourier space and pass it into the encoder to represent its structural heterogeneity in a distribution of some latent variable z . Sampling from this distribution, we obtain a sample of z - this, along with the decoder, is passed into the Branch & Bound algorithm to find the rotation R and the translation t . Using this, we create a positional encoding of a pixel; this, along with sampled values of our distribution of z , is then passed into our decoder. The distributions of density generated by the decoder are then subject to an inverse Fourier transform, the translation t , and the microscope CTF (Contrast Transfer Function), allowing us to reconstruct a slice of the image. When training, we aim to optimize

$$L(\hat{X}_i; \xi, \theta) = \mathbb{E}_{q_\xi(z|\hat{X}_i)} \left[\log p_\theta(\hat{X}_i|z) \right] - \text{KL}(q_\xi(z|\hat{X}_i) || p(z))$$

, which is similar to a standard VAE loss function. The first term represents the probability of reconstructing the image $\hat{X}_i$ given the sampled z while the second term corresponds to the KL divergence, ensuring that our distribution outputted by the encoder does not diverge greatly from the standard normal. By learning an optimal ξ (parameter of the encoder network) and θ (parameter of the decoder network), we ensure that our predicted distribution from the encoder learns a distribution of z for images $\hat{X}_i$ which is normalized and structured. Our predicted distribution for the decoder allows for optimal reconstruction of an image given some sampled z value.

3.2 Pre CryoDRGN

Since our overall goal was to implement a model that follows the novel architecture of the CryoDRGN model and reconstructs continuous distributions of 3D protein structure from 2D projections of 3D proteins, we have built up to the machine learning side goal by first implementing a Variational Autoencoder (VAE) and then a Spatial Variational Autoencoder (Spatial VAE).

We have tested our implementations of the VAE [4] and Spatial Variational Autoencoder on the Modified National Institute of Standards and Technology (MNIST) dataset [2], a collection of 70,000 grayscale hand-written digits from 0 to 9, each 28 x 28 pixels in size. For the data preprocessing for the VAE, we binarized the MNIST images. For the data preprocessing of the Spatial VAE, we added rotation and translation data transformations, creating various training datasets of rotated MNIST digits and translated MNIST digits. We subsample 1000 or 2000 images from each digit to build the training dataset. We do not resize the MNIST images.

The model architecture of our VAE consists of a multi-layer perceptron (MLP) with two linear layers for the encoder and an MLP with two linear layers for the decoder, as illustrated in Figure 3. Between the layers, ReLU activations are applied. The encoder transforms the input MNIST image x to a latent variable z by passing through the first linear layer that maps the input dimension to a hidden dimension and the second linear layer that maps the hidden dimension to a z dimension for latent variables μ and log variance. In other words, the encoder encodes the input x into a mean-vector μ and the log variance vector $\log \sigma^2$. To enable backpropagation through the stochastic latent variable, we apply the reparameterization trick. We perform the reparameterization trick by sampling from a standard normal distribution. Then we generate a latent variable z , based on the mean and the standard deviation from the encoder and on the sample. In the decoder, we transform the reparameterized latent variable z back to the input space, creating a reconstructed image. This is achieved through the first linear layer mapping the latent variable z to a dimension of the hidden dimension and the second linear layer mapping the hidden dimension to the dimension of the reconstructed image. The parameters we used to train the model were batch size 128, input z dimension 728, hidden dimension 256, and z dimension 50, and the Adam optimizer. To calculate the reconstruction loss, we used mean-squared error, as binary cross entropy error performed worse. To calculate the KL divergence loss, we use the following closed-form formula:

$$\text{KL Divergence Loss} = -\frac{1}{2} (1 + \log \sigma^2 - \mu^2 - \sigma^2)$$

We modified the general VAE to be a spatial VAE, to disentangle the content of the image from potential rotations and translations. In order to ensure that our spatial VAE was actually using the spatial grid information concatenated with z , we queried the decoder with random permutations of subsamples of spatial coordinates of each image.

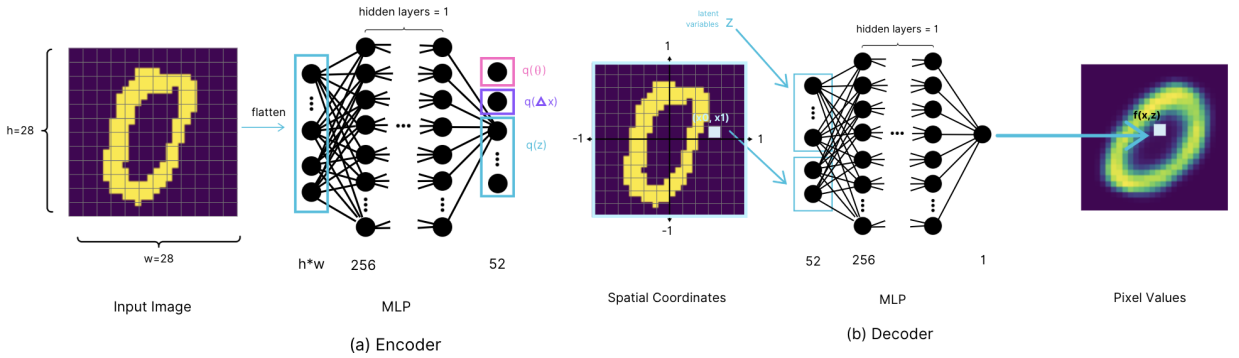


Figure 3: Diagram of the Spatial-VAE framework. a) The encoder model learns the rotation, translation, and unstructured latent variables using an inference network, depicted here as an MLP, as this is the architecture used in the experiments. However, the model can be generalized to other architectures as well. b) The encoder is an MLP mapping spatial coordinates and latent variables learned from the encoder to the distribution parameters of the pixel intensity at that coordinate. During training, inputting every spatial coordinate into the decoder is unnecessary. This is made possible by the disentanglement of rotation, translation, and unlabeled latent representations the encoder learns in the spatial VAE. During testing, the decoder is applied to each coordinate in the pixel grid to generate a complete image in the 2D space of the input image. Coordinate transformations (such as Fourier transform as explained in data visualizations) are applied to the spatial coordinates *before* being decoded by the generator network.

The primary differences between a spatial VAE and a standard VAE that allow the model to learn the pixel intensities at a location on the image independently of the other pixel locations are the loss function and decoder inputs that allow the latent representations of the rotation, translation, and other latent variables to be separated in the output of the encoder. Hence, the encoder architecture of the spatial VAE matches that of the VAE. The posterior is represented as $\log q(z, \theta, \Delta x | y) = \log \mathcal{N}(z, \theta, \Delta x; \mu(y), \sigma(y)^2 I)$, the prior for z is $z \sim \mathcal{N}(0, I)$, and the prior for Δx is $\Delta x \sim \mathcal{N}(0, \sigma^2)$. The prior for θ is $\theta \sim \mathcal{N}(0, \sigma^2)$, where the standard deviation of this distribution

is set to π to approximate a uniform prior over θ . This results in modification of the KL divergence loss for theta. The first index of z represents parameters corresponding to rotation, and the following two indices of z represent parameters corresponding to translation. The reparameterization trick is applied as in the VAE to generate z .

The input to the decoder in the spatial VAE varies from the input to the decoder in the VAE, as the decoder input is the latent variable z concatenated with a spatial coordinate (x_0, x_1) where the output of the decoder is the distribution of parameters of the pixel intensity at that coordinate. The spatial coordinates fed into the decoder are a 2D grid of normalized Cartesian coordinates (with the origin at the image center) of all coordinates in the image or a subsampling of pixels. Before concatenating the spatial coordinate to the latent variable z , if rotation or translation is enabled, transformations are directly applied to the spatial coordinate. If rotation is enabled, the spatial coordinates are multiplied by a rotation matrix where θ in the rotation matrix is from the generated z , and similarly, if translation is enabled, a shift by Δx occurs where Δx is from the generated z . The transformed latent variable z and the spatial coordinates are passed into the decoder, which maps the latent information back to the input space, producing a reconstruction of x . We used the parameters to train the model: batch size 64, input z dimension 728, hidden dimension 256, z dimension 50, and the Adam optimizer. To calculate the reconstruction loss, we used mean-squared error (and tested binary cross entropy error). To calculate the KL divergence loss for z and Δx , we use the following closed-form formula:

$$\text{KL Divergence Loss} = -\frac{1}{2} (1 + \log \sigma^2 - \mu^2 - \sigma^2)$$

To calculate the KL divergence loss for θ (to account for the fact that the KL divergence does not penalize the approximate posterior mean given a uniform prior), we use the following closed-form formula:

$$\text{KL Divergence Loss}_\theta = -\log \sigma_\theta + \log s_\theta + \frac{\sigma_\theta^2}{2s_\theta^2} - 0.5$$

The total KL divergence loss is the sum of the individual KL divergence losses. These are our modifications from the VAE model architecture and training process to the Spatial VAE mode architecture and training process.

To transition from MNIST images to CryoBench images, we note that the shapes of the photos are 128×128 instead of 28×28 . We considered either resizing to shape 28×28 or passing in 128×128 as the model is resolution-independent.

3.3 3D Reconstruction

Given 2D cryoem images, we can apply a Fourier transform to move them to the Fourier space and then apply the Fourier Slice Theorem to obtain a 3D volume in Fourier space, which we can then inverse the Fourier transform to get our desired 3D volume. How can we form a 3D volume from 2D slices? Before delving into the application of the Fourier Space Theorem with our research methods, we will demonstrate a 3D reconstruction from 2D projection images previously generated. This will be a stepping stone experiment conducted before 3D reconstructing with the slices from our model. Our 3D reconstruction method is as follows:

1. Given projections* (shape D x D), move them to the Fourier space via a 2D Fourier transform.
2. Assuming known (or estimated) rotations, take a central plane ($z = 0$) of a D x D x D empty grid and apply rotation R to obtain the original coordinates of that plane.
3. Use the nearest-neighbor interpolation method to choose the closest integer grid point.
4. Each projection will have its array of nearest-neighbor grid points (shape D * D x 3).
5. Focus on the grid. For each point in the grid, if it is in an array corresponding to a slice, add the value at the point in the slice to the point in the grid.
6. Normalize by division by weights (calculated by the number of times a grid point was updated).

* Note that in our model, we would begin with the slices from our model output. Since slices of 3D Fourier space volume and Fourier spaced 2D projections are equivalent by the Fourier slice theorem, this is a valid method of testing 3D reconstruction before application in our model.

We further investigated the quality of 3D reconstructions depending on input projection dimension D and the number of projections used for reconstruction.

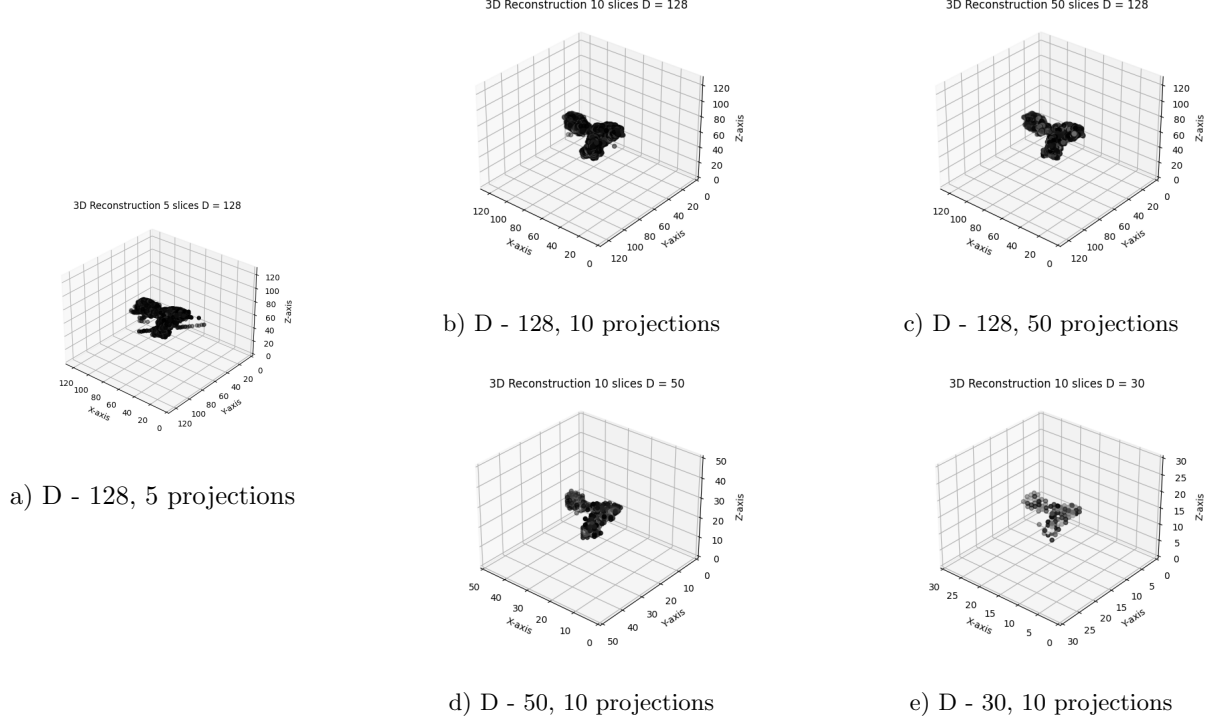


Figure 4: Comparison of 3D reconstructions with different numbers of projections and slices. We analyze combinations of projection image dimensions with a number of projections used to 3d reconstruct to find the optimal choice with the best reconstruction. We find that for $D = 128$, b) 128 D 10 projections reconstruct the best with the fewest slices used compared to a) 128 D 5 projections and c) 128 D 50 projections. 10 projections will be our choice for the fewest projections used with competitive reconstruction. d) 50 D 10 projections and e) 36 D 10 projections compare projection dimension size, where we resize the projection image using the OpenCV package. e) reconstruction suffers from low D , so if we wanted to limit D parameter size, $D = 50$ would be the optimal choice.

3.4 Metrics

The primary metric we use to evaluate our implementation of CryoDRGN is the commonly used Fourier Shell Correlation (FSC) metric [8]. The FSC is a criterion that evaluates the reconstruction quality of methods such as CryoDRGN at 3D reconstruction. We apply this metric to the 3D reconstructions obtained by our model to the ground truth volume.

The Fourier Shell Correlation is applied onto two volumes in the Fourier domain $\hat{U}$ and $\hat{V}$. The criterion measures the correlation of the two at a spherical shell S_k , defined as a set of Fourier voxels located at a specific radial distance k from the origin. The numerator, $\sum_{s \in S_k} \hat{U}_s \hat{V}_s^*$, captures the total correlation between $\hat{U}$ and $\hat{V}$ across all voxels in shell S_k , while the denominator, the square root of the self-correlation of $\hat{U}$ and $\hat{V}$, serves as a normalizer, ensuring that the FSC ranges from -1 to 1. 1 indicates perfect correlation within the shell, 0 indicating no correlation, and negative values suggesting an inverse relationship. $\hat{V}_s^*$ denotes the complex conjugate of $\hat{V}_s$.

$$\text{FSC}(k) = \frac{\sum_{s \in S_k} \hat{U}_s \hat{V}_s^*}{\sqrt{\left(\sum_{s \in S_k} |\hat{U}_s|^2\right) \left(\sum_{s \in S_k} |\hat{V}_s|^2\right)}}$$

4 Results

We present qualitative and quantitative results for homogeneous cryo-EM reconstruction, demonstrating how effectively the volumes generated using our implementation of the cryoDRGN pipeline align with those produced by established methods.

4.1 Supervised Homogeneous Reconstruction

4.1.1 Training with Noiseless Projections

We evaluate our cryoDRGN model (CryoDRGN VAE) on a homogeneous dataset of 2D projections. Specifically, we train our model on a custom-generated synthetic dataset as described in section 2.1 based on the cryo-EM image formation model, consisting of images with dimension $D = 50$ and a total of 1,000 projections with noise. [3].

While the decoder network is adequate for homogeneous reconstruction, we present results using the image-encoder volume-decoder network to empirically illustrate its potential for extending to the heterogeneous reconstruction scenario. Thus, the 2D slices generated by the model are derived from a pose-invariant latent representation corresponding to the input projection provided to the model.

As demonstrated in Figure 5, our model successfully reconstructs 3D protein structures from noiseless 2D projections of a volume with pose supervision, performing comparably to existing methods.

To confirm that our model’s decoder learns to reconstruct the correct slice at the corresponding spatial coordinates during training, we evaluate its performance under two conditions: (1) providing the decoder with the actual rotation of the volume that matches the input projection and (2) supplying a random rotation. Figures 8, 9, 10, and 11 illustrate that the model fails to generate accurate slices when queried with incorrect coordinates that do not correspond to the actual 3D location of the input slice. This finding supports the conclusion that the decoder learns to produce the electron density *at* the queried spatial coordinate rather than simply replicating a slice that visually resembles the input projection but originates from an incorrect 3D region. Furthermore, Figure 12 demonstrates that the decoder can be queried across spatial coordinates to reconstruct the entire electron density volume of the protein, emphasizing that the model learns to utilize the input coordinates effectively rather than relying on memorizing slice views without considering their 3D spatial context.

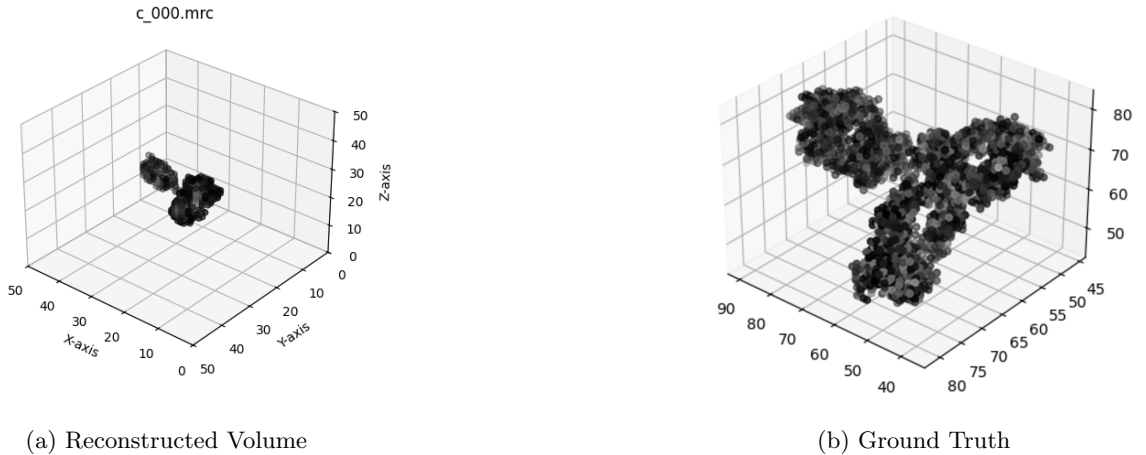


Figure 5: (a) The 3D protein reconstruction of the IgG antibody complex (when trained on noiseless projections of the ground truth, sampled at random orientations) and (b) the ground truth volume. Reconstruction similarity to the ground truth was evaluated using the Fourier Shell Correlation (FSC) resolution metric, shown in Figure 7.

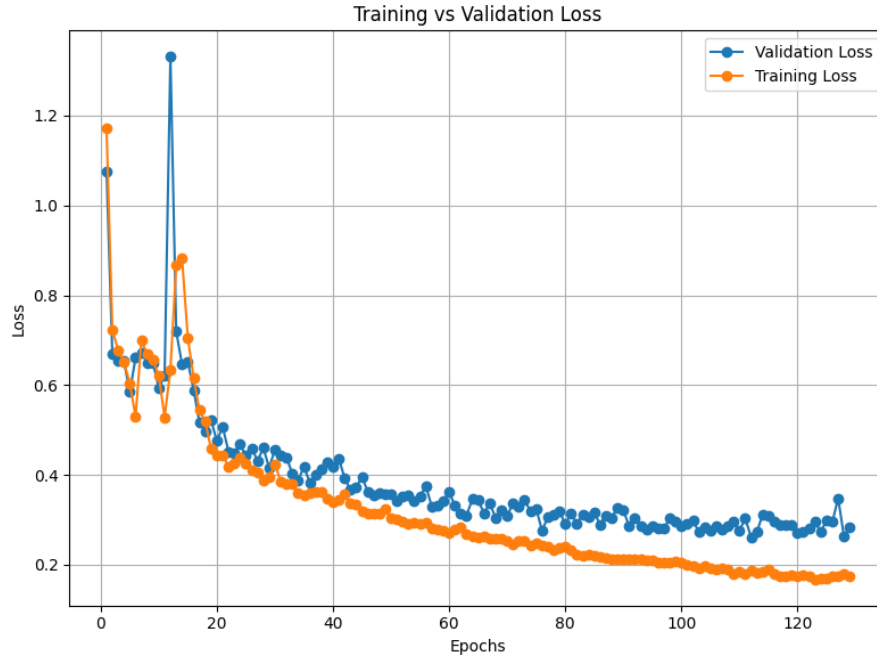


Figure 6: Training vs Validation Loss Plots corresponding to Reconstructed Volume from Figure 5

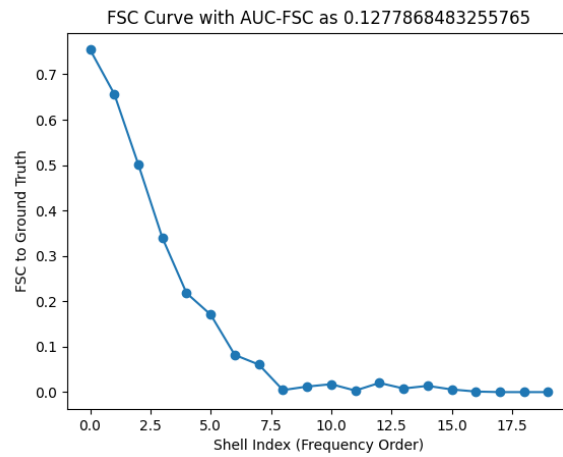


Figure 7: FSC Values Computed for Output - Ground Truth corresponding to Figure 5. We see that there is strong agreement or correlation for the broad features of the structures, as demonstrated by a high FSC value for lower shell indices.

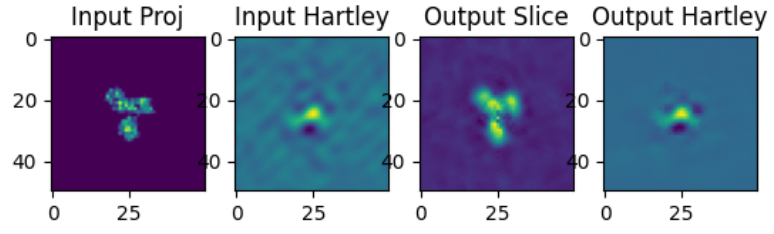


Figure 8: A 2D slice inferred by cryoDRGN using the correct rotational alignment.

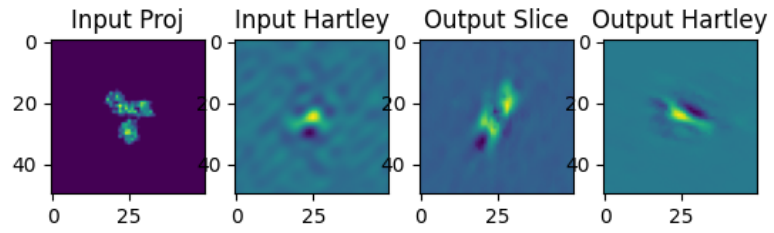


Figure 9: A 2D slice inferred by cryoDRGN using a random rotational alignment.

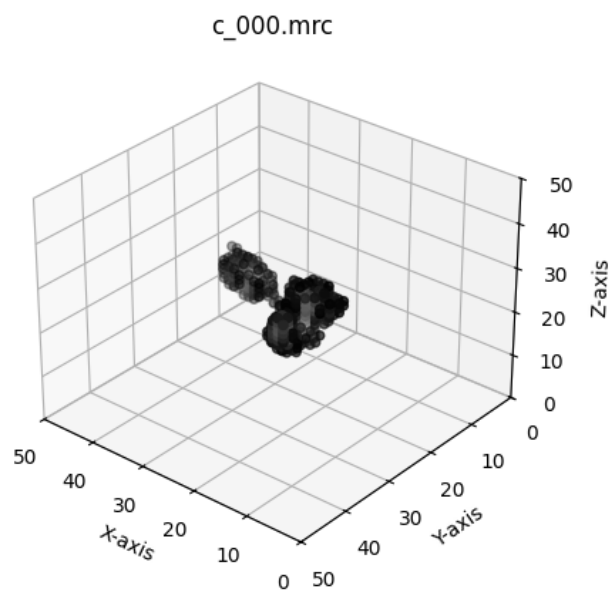


Figure 10: 3D reconstruction generated by cryoDRGN using a test dataset with correct rotational alignments provided.

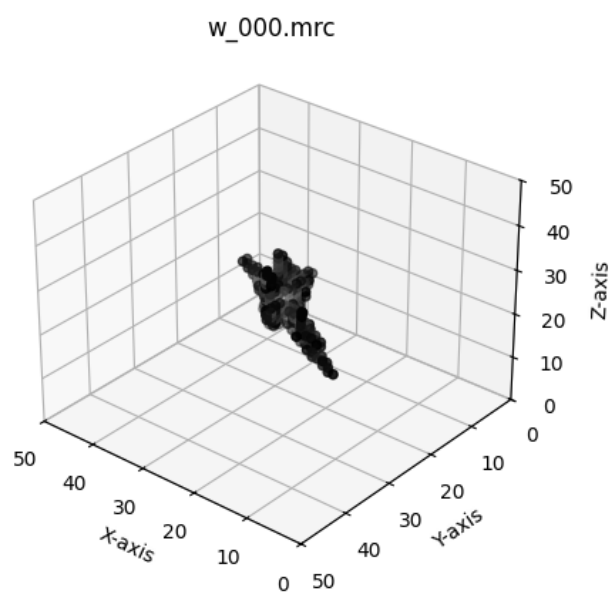


Figure 11: 3D reconstruction generated by cryoDRGN using a test dataset with random rotational alignments provided.

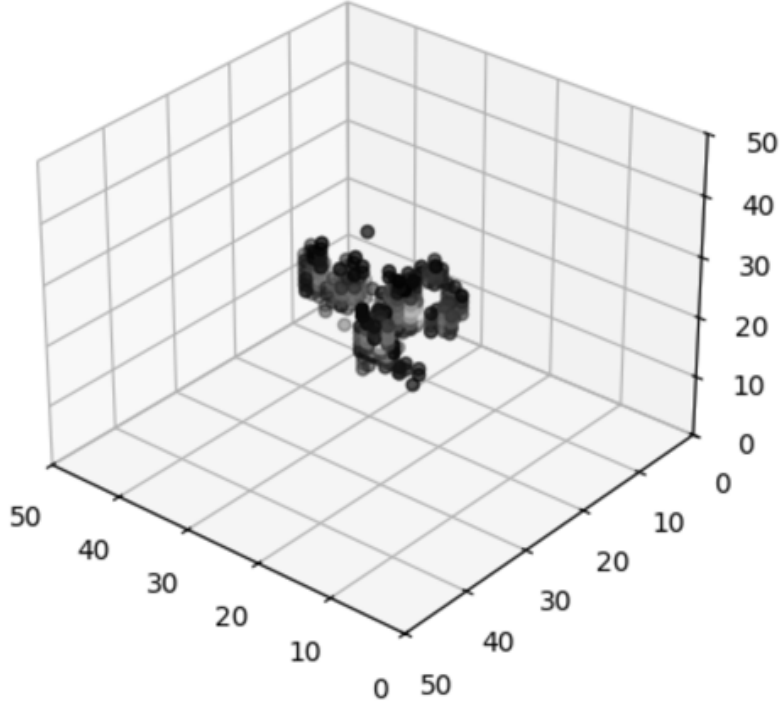


Figure 12: 3D reconstruction generated by decoder.

4.1.2 Training with Noisy Dataset

We also train our model on noisy images from a CryoBench dataset, specifically the IgG-1D signal-to-noise ratio (SNR) 0.01 images, consisting of 1,000 2D projection images corresponding to randomly oriented volumes. These images were derived by rotating one domain of the IgG antibody complex through a complete 360-degree cycle, effectively simulating simple, one-dimensional continuous circular motion [3].

However, training on noisy cryoBench data (SNR 0.01) significantly hinders the model’s ability to denoise input images and extract meaningful structural information, instead of learning features dominated by noise (Figure 13). Future work could address this limitation by incorporating a positional encoding that excludes the highest frequency components of the noiseless positional encoding, thereby enhancing the model’s robustness to noisy inputs.

To address the challenges our model faces with noisy images, we experimented with various image preprocessing techniques to mitigate both random Gaussian noise and noise introduced by the microscope imaging process. The filters applied are summarized in Figure 14.

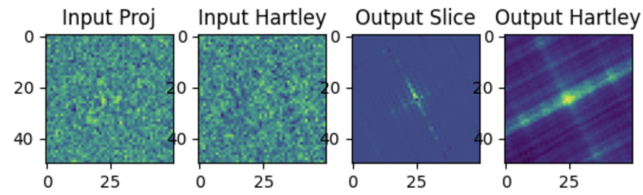


Figure 13: 2D slice generated by the cryoDRGN model during inference, sampled at a rotated lattice, after being trained on a noisy cryoBench dataset with an SNR of 0.01.

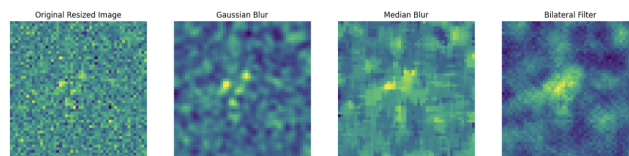


Figure 14: Denoising Filter. *Left:* The original input image. *Right:* Gaussian, Median, and Bilateral blur filters are applied to the original input image.

4.2 Unsupervised Homogeneous Reconstruction

We modified cryoDRGN’s architecture to explicitly encode pose information by ensuring that the latent variable z learned by the encoder captures the orientation of the ground truth volume corresponding to the input image. The spatial VAE inspired this modification. We used the rotation matrix obtained from the components of the sampled latent representation z to rotate the lattice before passing those points into the decoder. The reasoning for this model is further described in Methods. As illustrated by Figure 15, the model failed to reconstruct a 3D protein structure properly in this attempt. When comparing the predicted rotation matrices against the ground truth rotation matrices, it can be seen that the learned rotation matrices varied significantly even after long periods of training. This suggests that the network failed to properly disentangle pose information from other features and that it can be difficult to encode pose in specific z components. An alternative approach to learning the protein pose (rather than directly mapping pose to specific latent components) was to use an MLP, where the input is the latent variable z and the output is a quaternion. This approach yielded similar results. In both methods, the model failed to learn the rotations correctly. The output seen in Figure 15, appears symmetric likely due to the fact that incorrectly learning rotation is analogous to randomly averaging over the rotations. This likely occurs because the model is effectively averaging over many incorrect rotations. Random or inconsistent rotations result in reconstructed structures that lose directional information and resemble a rotational average. Based on these results it can be concluded that it is difficult to learn pose in this model architecture.

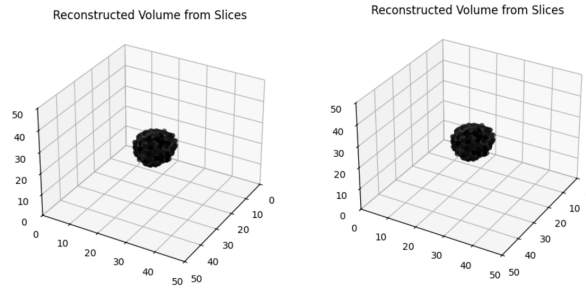


Figure 15: 3D Reconstruction Attempts from Spatial VAE. *Left*: Reconstruction/output of the direct encoding of the pose in the latent z . *Right*: Reconstruction/output of the quaternion through a MLP. The FSC scores for the lefthand reconstruction are -0.01206111 and 0.25219871, and -0.02384308 and 0.25377261 respectively. The ground truth volume is displayed in Figure 5

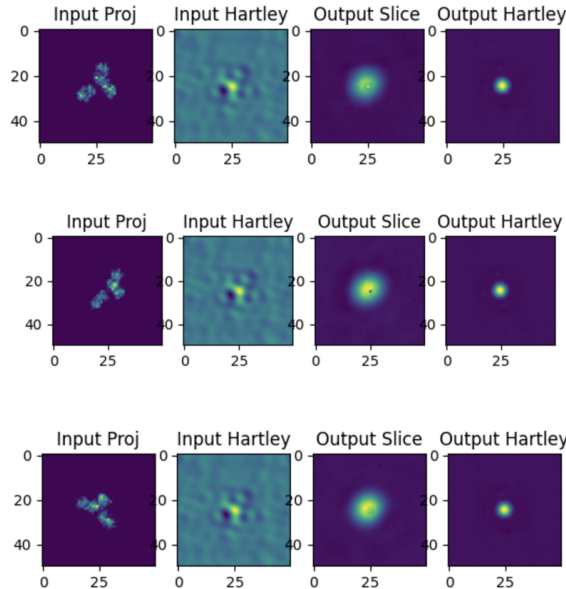


Figure 16: 2D Slices from 3D Reconstruction Attempt from Spatial VAE with pose explicitly encoded in z

5 Future Work

In the original CryoDRGN model architecture, to find the pose of an image (specifically the rotation R and the translation t), a Branch & Bound algorithm is used in CryoDRGN. A global search over $SO(3) \times \mathbb{R}^2$ (where

$SO(3)$ refers to a special orthogonal group in 3D and represents all orientations about the origin in 3D space) is conducted. This is done with a sampled value z from the distribution and the decoder (which allows us to measure the likelihood of an image given a certain rotation and translation, aiding in finding the optimal pose). The Branch and Bound Algorithm begins by discretizing the search space onto a uniform grid, where each point represents a pose. The criteria for pruning (i.e. terminating certain paths) is based on a computationally inexpensive lower and upper bound of the objective (which is the likelihood of the pose). If a grid point has a higher lower bound than the predefined computational lower bound, it is discarded. Another optimization involves discarding high-frequency components, which do not contain significant information about the image - therefore, not using those components allows the algorithm to focus on the informative parts of the image as lower-frequency components capture most of the accuracy. As the search progresses, the frequency bound increases, allowing for more nuances in the image to be accounted for in the search for the correct pose and translation. One of the limitations of the Branch & Bound Algorithm is that it can be very time-consuming and computationally expensive, which leads Branch & Bound to be a bottleneck in the training algorithm as a lot of iterations and memory are required to find the optimal solution when the enumeration can not be done in a reasonable time. The Branch and Bound algorithm also involves heuristic methods. In order to train a model to learn protein pose when protein poses are unknown in an unsupervised nature, we implemented an extension of the spatial VAE model as an alternative approach that can be used to the Branch & Bound algorithm to find the pose of a protein. Unlike a regular VAE, spatial VAEs associate a positional encoding with the image such that it learns to reconstruct an image with explicit consideration of spatial positions, allowing us to separate the structural heterogeneities from rotational and translational factors and better understand the structural information at each pixel. Under the spatial VAE approach, the pose is encoded in a component of the latent variable z . Then, the designated components of the latent variable z that correspond to the learned pose are separated and are used to rotate the input slice grid points into the decoder. The mapping allows the model to encode both the visual features and the spatial configuration of the protein. One way to encode pose in the latent variable z is through the use of a Spatial VAE is less computationally expensive and involves less heuristic methods than the Branch and Bound algorithm. Our results conclude however that the suggested alternatives are unable to serve as a replacement to the Branch and Bound algorithm.

References

- [1] Tristan Bepler, Ellen Zhong, Kotaro Kelley, Edward Brignole, and Bonnie Berger. Explicitly disentangling image content from translation and rotation with spatial-vae. *Advances in Neural Information Processing Systems*, 32, 2019.
- [2] Li Deng. The mnist database of handwritten digit images for machine learning research [best of the web]. *IEEE signal processing magazine*, 29(6):141–142, 2012.
- [3] Minkyu Jeon, Rishwanth Raghu, Miro Astore, Geoffrey Woollard, Ryan Feathers, Alkin Kaz, Sonya M Hanson, Pilar Cossio, and Ellen D Zhong. Cryobench: Diverse and challenging datasets for the heterogeneity problem in cryo-em. *arXiv preprint arXiv:2408.05526*, 2024.
- [4] Diederik P Kingma. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013.
- [5] Jacqueline LS Milne, Mario J Borgnia, Alberto Bartesaghi, Erin EH Tran, Lesley A Earl, David M Schauder, Jeffrey Lengyel, Jason Pierson, Ardan Patwardhan, and Sriram Subramaniam. Cryo-electron microscopy—a primer for the non-microscopist. *The FEBS journal*, 280(1):28–45, 2013.
- [6] Amit Singer. Mathematics for cryo-electron microscopy. In *Proceedings of the International Congress of Mathematicians: Rio de Janeiro 2018*, pages 3995–4014. World Scientific, 2018.
- [7] A Vaswani. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017.
- [8] Eric J Verbeke, Marc Aurèle Gilles, Tamir Bendory, and Amit Singer. Self fourier shell correlation: properties and application to cryo-et. *Communications Biology*, 7(1):101, 2024.
- [9] Ellen D Zhong, Tristan Bepler, Bonnie Berger, and Joseph H Davis. Cryodrgn: reconstruction of heterogeneous cryo-em structures using neural networks. *Nature methods*, 18(2):176–185, 2021.

6 Appendix - Pre-CryoDRGN Results

We conducted a series of experiments to evaluate the capacity of the Spatial VAE to learn and reconstruct structural details of 2-D MNIST images of digits by training on various sub-samples of spatial coordinates. Our results demonstrate that the model effectively reconstructs training and testing images under a variety of transformations—unaltered, rotated, translated, and both rotated and translated—as long as the spatial coordinates passed into the decoder during training are contiguous. The posterior distributions of the first and second latent variables (notably θ , representing orientation) appear approximately jointly normally distributed, indicating that the model is successfully capturing relevant spatial information within the latent space. Training and validation losses consistently decrease with an increasing number of epochs, which is a positive indicator of learning stability.

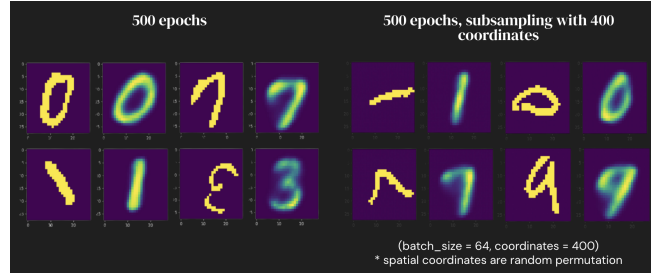


Figure 17: The original image (left) and the image reconstructed by the Spatial VAE (right). The reconstructed images appear to be a similar shape as the original image, with added noise, as the decoder is only trained on a subsample of spatial coordinates in the original image, which decreases the accuracy of the reconstruction loss.

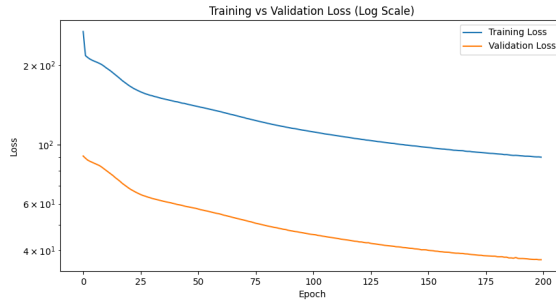


Figure 18: The training and validation Loss for the Spatial VAE associated with the rotated dataset of MNIST images. Both losses decrease as the number of epochs increases, and the validation loss does not increase as the number of epochs increases so we are less vary of over-fitting.

We experimented with training the decoder model on various contiguous subsamples of spatial coordinates. Our results yielded reasonable reconstructions, albeit with lower resolution than the original images. However, when random subsamples of spatial coordinates were passed into the decoder during training, the resulting reconstructions appeared notably noisy. In this case, we found that the reconstruction loss was significantly high relative to the KL divergence; we will experiment with the down-weighting factor $\alpha = 0.02$ applied to the KL divergence term, potentially increasing it. This suggests the need for further optimization in the loss function, such as verifying the approach used to obtain the pixel intensities of the original images corresponding to the randomly chosen coordinates during the subsampling process, as well as analyzing the subsampling effect on the encoder and decoder models as they are trained together.

In optimizing the Spatial VAE, we evaluated a variety of configurations. While experimenting with different loss functions, we found that the Mean Squared Error (MSE) outperformed Binary Cross-Entropy (BCE) as it produced reconstructions with higher resolution and clarity. We also ran experiments with varying latent dimensions, hidden layer sizes, and activation functions. Most experiments yielded similar outcomes, given that MNIST is a relatively simple dataset for reconstruction. Hence, we present our results with a latent dimension of 52, ReLU activation, 256 hidden layers, and 500 subsampled spatial coordinates (for training decoder), as further explained in the methods section. Collectively, our results underscore the Spatial VAE’s potential for learning to reconstruct structures without having seen every spatial coordinate during training.